HacdsDeFi RPC 文档

2026-11-11　对应合约 #3000　面向开发者

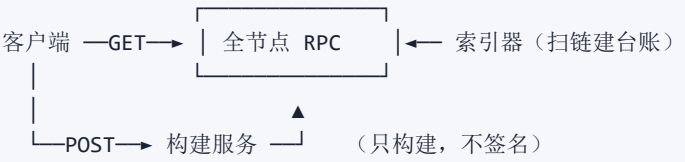
> 本文档描述 Hacash 主网合约 `HacdsDeFi3000.fitsh`
> （地址 `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`，资产 serial 3000，decimal 0）。
>
> **本文档不含任何 #2000 时代的接口。** 预言机、评分证书、管理员函数在新版合约里**已不存在**——不是权限被收走，是这些函数根本没写进去。

---

0. 服务一览

hacds 涉及四个 HTTP 服务。前两个读链，后两个是便利设施。

服务	地址	协议	作用
全节点 RPC	`node.hacds.com`	HTTP/JSON	查链上数据、沙箱只读调用、校验/广播交易
API 代理	`api.hacds.com`	HTTP/JSON	转发到全节点，补 CORS + HTTPS
索引器	`indexer.hacds.com`	HTTP/JSON	借款人台账（谁可清算）
交易构建服务	`build.hacds.com`	HTTP/JSON	把人类输入转成未签名交易



为什么需要 API 代理：全节点不返回 CORS 头，浏览器直连会被拦。

`api.hacds.com` 是一个 Cloudflare Worker，只做转发和补头，不改语义。

服务端端口不是 80/443，Cloudflare 到源站走 Tunnel。

响应约定：所有接口返回 JSON，顶层带 `ret` 字段。`ret: 0` 表示成功，

非 0 表示失败（失败时通常带 `err`）。

> ⚠ **失败也是 HTTP 200**，必须判 body 里的 `ret`，不能只看状态码。

为什么节点地址是子域名而不是 `IP:端口`：Cloudflare Workers 只能访问

80/443 端口。任何 `IP:8082` 形式的地址都会在建立连接前被拒绝

（`error code: 1003`）。所以对外统一走 `*.hacds.com` 子域名 + Tunnel。

---

1. 全节点 API

1.1 查询区块高度

```
GET /query/block/height/latest
```

```
{"ret":0,"height":798812}
```

VM 状态（是否已启用、当前高度）：

```
GET /api/vm/status
```

```
{"ret":0,"vm":"enabled","height":798812}
```

1.2 查询平均费率

用于估算手续费。

```
GET /query/fee/average
```

```
{"ret":0,"purity":"6024000000","purity_unit":232}
```

`purity` = `fee / billing_size（字节）`，节点准入要求 `纯度 ≥ min_purity`。

1.3 查询余额（HAC / 钻石 / 资产）

```
GET /query/balance?address={地址}&assets=true&asset_meta=true&diamonds=true
```

三个开关都是可选的：

参数	作用
`assets=true`	带上资产列表（hacds 在这里）
`asset_meta=true`	带资产名 / 发行量等元信息
`diamonds=true`	带上 HACD 钻石列表

```
{
  "ret": 0,
  "list": [{
    "address": "1P2tsn...",
    "hacash": "1234567:248",
    "diamond": 3,
    "assets": [{"serial": 3000, "amount": "100000000"}]
```

```
  }]
}
```

金额是 `<值>:<单位>` 形式：`5:249` 表示 50 HAC（5 × 10^(249−248)），

不是 5.0038。换算公式见 §6.3。

1.4 查询钻石详情

```
GET /query/diamond?name={6位钻石名}
```

返回钻石的编号、视觉基因、所属地址等。评分需要的 `number` 和

`visual_gene` 都在这里。

1.5 沙箱只读调用（读合约状态的主要方式）

```
GET /query/contract/sandboxcall?contract={合约}&function={函数}&params={参数}&caller={调用者}
```

只读执行，不写链上状态、不发交易、不花手续费。

实测响应：

```
{"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}
```

字段	含义
`ret`	0 = 调用成功
`ret_val`	函数返回值
`use_gas` / `gas_use`	本次消耗，用于估算

> ⚠ **合约部署前，这里会返回 `NotFindContract`。** 这是正常的：
> 合约要到高度 800,000 才部署。部署后即可调用。

1.5.1 两个必知的坑

坑 1：不传 `caller` 会失败。

```
{"ret":1,"err":"[revert] address \"1111...\" balance 0:0 is insufficient, at least 559555:242"}
```

默认调用者余额为 0，付不起预扣 gas。**传 `caller={合约地址}` 即可**——

合约调用自己不需要余额。

坑 2：每 IP 并发上限为 2。 超出的请求**直接拒绝**，不排队：

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

客户端必须**串行化 + 退避重试**。不要 `Promise.all` 并发发十几个。

1.5.2 参数格式

`params` 是 `值:类型` 用逗号分隔。`:` 要 URL 编码成 `%3A`。

类型	示例
`u128`	`100000000:u128`
`address`	`1P2tsn...:address`
`bytes`	`WTYUIA:bytes`

1.5.3 只读函数完整清单

AMM 与流动性

函数	参数	返回	说明
`amm_reserves`	—	u128	池中 HAC（**zhu**）
`amm_reserve_ds`	—	u128	池中 hacds（**个数**）
`pool_price`	—	u128	池价 = `amm:hac × 1e8 / amm:ds`
`lp_total`	—	u128	LP 总份额
`lp_share_of`	`who:address`	u128	某地址的 LP 份额
`lp_pending_hac`	`who:address`	u128	待领 LP 奖励（HAC，zhu）
`lp_pending_ds`	`who:address`	u128	待领 LP 奖励（hacds，个数）
`total_staked`	`kind:u128`	u128	kind `0`=HAC，`1`=hacds

借贷

函数	参数	返回	说明
`loan_debt_of`	`who:address`	u128	本金（zhu），**不含**未结算利息
`loan_debt_now`	`who:address`	u128	含利息的实时债务（zhu）
`loan_coll_of`	`who:address`	u128	锁定抵押（hacds 个数）
`loan_health`	`who:address`	u128	**健康度万分比**，见下
`loan_status`	`who:address`	u128	抵押率 ×100，见下
`total_debt`	—	u128	全局债务（zhu）
`providers_total`	—	u128	全部可出借资金
`utilization`	—	u128	当前利用率（bps）
`borrow_coll_ratio`	—	u128	借款线百分比（`200`）

函数	参数	返回	说明
`liq_coll_ratio`	—	u128	清算线百分比（`160`）
`borrow_paused`	—	u128	非 0 = 暂停新增借款（利用率或熔断）
`ref_paused`	—	u128	非 0 = 价格偏离熔断中
`lendable_amm`	—	u128	AMM 可出借部分（30%）
`net_claims`	—	u128	全网债权
`reserve_hac`	—	u128	坏账储备
`deficit_hac`	—	u128	赤字

健康度的正确读法

`loan_health` 返回	含义
`1000000`	无债务（合约返回的哨兵值，视为极健康）
`> 10000`	安全
`= 10000`	恰在清算线上
`< 10000`	**可被清算**
`< 9000`	深水区，可被全额清算

关键：这个值用 `coll_to_hac_liq()` 计算，即**保守价** `min(现货价, 平滑参考价)`，

与 `loan_liquidate` 的判定**同源**。所以不会出现「界面显示安全、链上已可清算」。

不要用 `loan_status` 自己算健康度，用这个。

`loan_status` 是旧口径（`coll_hac × 100 / debt`，用现货价），保留兼容：

返回	含义
`0`	无借贷、无抵押
`999`	有抵押但无债务
其它	抵押率 ×100

HACD 池

函数	参数	返回	说明
`hacd_score_total`	—	u128	池内 HACD 总分（已 × `SCORE_SCALE`）
`hacd_count`	—	u128	池内 HACD 颗数
`hacd_avg_score`	—	u128	平均分（已 × `SCORE_SCALE`）
`hacd_coverage`	—	u128	覆盖率（bps，10000 = 100%）

函数	参数	返回	说明
`quote_redeem`	`sc:u128`	tuple	赎回报价，见下
`quote_score`	`name:bytes,vg:bytes,num:u128`	u128	按名字+基因+编号算分

`quote_redeem` 返回值：`tuple(charge, fee, premium_bps, ok)`

元素	含义
`charge`	总扣减（本金 + 手续费）
`fee`	其中手续费
`premium_bps`	挑钻石溢价（bps）
`ok`	`1` = 可赎回；`0` = 实际交易会返回 `61`（池内无记录或分数不足）

价格与因子

函数	参数	返回	说明
`ref_price_of`	—	u128	平滑参考价
`ref_price_zhu_per_ds`	—	u128	参考价（zhu per hacds）
`ref_height`	—	u128	上次更新参考价的块高
`hcut`	—	u128	HAC 社会化因子（1e12 定标，只降不升）
`lcut`	—	u128	LP 奖励社会化因子

账户与金库

函数	参数	返回	说明
`balance_hac`	`who:address`	u128	有效 HAC 余额（zhu，已乘 hcut）
`balance_hac_nominal`	`who:address`	u128	名义份额（zhu）
`balance_ds`	`who:address`	u128	待领取 hacds 配额 due
`bad_of`	`who:address`	u128	该地址的坏账
`treasury_pending`	`kind:u128`	u128	金库待清扫。`0`=HAC，`1`=hacds
`treasury_ds`	—	u128	金库 hacds 累积
`tot_hac`	—	u128	HAC 总量（有效值）
`tot_hac_nominal`	—	u128	HAC 总量（名义份额）
`lpr_hac_effective`	—	u128	LP 奖励池有效值

运维

函数	参数	返回	说明
`self_check`	—	u128	内部一致性自检，`0` = 通过
`rent_key_group_count`	—	u128	键分组数量（固定 `7`）
`rent_key_len_at`	`idx:u128`	u128	第 idx 组的键长度
`rent_keys_group`	`klen:u128`	bytes	该长度组的所有键（可直接喂 `rent_keys`）

1.5.4 实测样本

```
GET {node}/query/contract/sandboxcall?contract=T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH
    &function=loan_health&params=1P2tsn...%3Aaddress&caller=T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH
→ {"ret":0,"use_gas":830,"gas_use":{"compute":363,"resource":323,"storage":144},"ret_val":1000000}
```

`ret_val: 1000000` = 该地址无债务。

1.6 查询区块概要

索引器用它遍历区块找借款人。

```
GET /query/block/intro?height={高度}&tx_hash_list=true
```

1.7 查询交易

```
GET /query/transaction?hash={交易哈希}
```

1.8 校验交易（附带人工可读描述）

```
POST /util/transaction/check
Content-Type: application/json

{"tx": "040000...（hex）"}
```

成功时返回解析结果和可读的动作描述，例如：

```
{"ret":0,"describe":"[CallContract] T1A8... -> swap_hac_to_ds(...)"}
```

这个接口不签名、不广播，纯校验。 钱包在广播前拿它做最后一道把关。

1.9 广播交易

```
POST /submit/transaction
Content-Type: application/json
```

```
{"tx": "040000...（hex）"}
```

```
{"ret":0}
```

> ⚠ `ret: 0` **只表示进了内存池**，不等于上链。出块目标 300 秒，
> 要按出块节奏回查 `/query/transaction`。

1.10 合约存储费参数

```
GET /query/contract/storage_cost?contract={合约}
```

用于估算续租成本。

---

2. 索引器 API

索引器扫链，维护「谁有债务、谁可清算」的台账。**钱包不本地扫链。**

2.1 健康检查

```
GET /health
```

```
{"ok": true, "service": "hacds-indexer", "head": 798812, "watched": 0}
```

2.2 台账快照（客户端唯一需要的接口）

```
GET /snapshot
```

返回全部被监视地址的健康度快照：

```
{
  "head": 798812,
  "updated": 1762900000,
  "items": [
    {"addr": "1P2tsn...", "debt": "100000000", "coll": "300000000",
     "health": 16000, "ratio": 160, "liq": true}
  ]
}
```

字段	说明
`debt`	`loan_debt_now`（含利息）
`coll`	抵押 hacds 个数

字段	说明
`health`	万分比，与 `loan_health` 同口径
`ratio`	抵押率百分比
`liq`	`true` = 当前可清算

2.3 仅快照（不触发同步）

```
GET /snapshot?cached=1
```

2.4 立即同步

```
POST /sync
```

强制扫到最新块。用于运维，正常不需要调。

2.5 监视名单维护

```
POST /watch/add?addr={地址}
POST /watch/remove?addr={地址}
```

索引器会自动发现借款人，这两个接口用于手工补漏。

2.6 积分覆盖率监控

```
GET /backing
```

对比「合约记的 due 总量」与「链上实际发行的 hacds」，用于发现异常。

2.7 续租批次

```
GET /rent_batches
```

返回按长度分组、可直接喂 `rent_keys` 的键批次。keeper 用它续租。

2.8 部署

```
python3 hacds_indexer.py --node http://127.0.0.1:8082 \
    --contract T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH --port 8124
```

---

3. 交易构建服务

这个服务把「人类可读的参数」翻译成「未签名交易」。**它不持有任何私钥， 构建出来的交易必须由客户端自己签名。**

源码：`hacds_exchange/indexer/hacds_build_server.py`

3.0 为什么需要它

Hacash 的交易构造需要精确的 fee / gas / 动作编码，还要处理钻石名列表的 wire 格式。让每个客户端自己实现一遍，就是在保证「三端行为不一致」。

把构造逻辑集中到一个服务，客户端只负责签名——**签名的东西变了，但签名的人没变。**

3.1 健康检查

```
GET /health
```

```
{"ok": true, "service": "hacds-build", "version": "2026-11-11.2",
 "txbuild_ok": true, "contract": "T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH"}
```

`version` 很重要：Python 不会热加载，改了脚本必须重启进程。发现行为和 `version` 对不上，先确认进程重启过。

3.2 构建合约调用

```
POST /api/build
Content-Type: application/json

{
  "mode": "call",
  "main": "1P2tsn...",
  "fn": "swap_hac_to_ds",
  "params": "100000000:u128,0:u128,0:u128",
  "fee": "0:248",
  "gasMax": 99
}
```

字段	必填	说明
`mode`	否	`call`（默认）或 `transfer`
`main`	✅	发起地址（会付手续费的那个）
`fn`	✅	合约函数名，白名单见 §3.4
`params`	否	`值:类型` 逗号分隔，默认 `-`
`fee`	否	形如 `0:248`
`gasMax`	否	整数 `1–99`，默认 `99`

响应：

```
{"ok": true, "unsigned_tx": "040000...", "fn": "swap_hac_to_ds",
 "fee": "0:248", "gas_max": 99}
```

3.3 构建转账

```
POST /api/build
{
  "mode": "transfer",
  "main": "1P2tsn...",
  "sub": "hacd-from-to",
  "fee": "0:248"
}
```

`sub` 是 `hacds_txbuild transfer` 的子命令，额外参数按子命令给出。

`from` **锁定为合约地址**——这是安全约束，防止有人用这个服务构造「从任意地址转出」的交易。

3.4 函数白名单

```
swap_hac_to_ds · swap_ds_to_hac
add_liquidity · remove_liquidity · lp_harvest
loan_add_collateral · loan_borrow · loan_repay · loan_liquidate
rent_keys · topup_keys
```

不在白名单里的函数会被拒绝。**这是白名单不是黑名单**——新增函数必须显式加入。

3.5 限制

限制	值	为什么
单 IP 每分钟请求	30	防滥用
请求体	8 KB	参数串上界 1024 字节、最多 16 项
钻石名列表	最多 200 颗	对应 `MAX_DEPOSIT_COUNT`
`gasMax`	1–99	合约/引擎合法范围

3.6 错误码

`error`	含义
`bad_json`	请求体不是合法 JSON
`bad_length`	请求体为空或超 8 KB
`bad_main`	`main` 不是合法地址
`bad_fee`	`fee` 不是 `值:尾数` 形式
`bad_gasMax`	`gasMax` 不是 1–99 的整数
`fn_not_allowed`	函数不在白名单
`bad_params`	参数串格式错

error	含义
rate_limited	超过每分钟 30 次
build_timeout	引擎超时（30 秒）
engine_missing	服务器上找不到 hacds_txbuild

---

4. 交易所上币对接

4.0 先记住一件事：amount 是整数个数

hacds 是 **decimal 0** 的资产。链上返回的 amount 就是个数，**不做任何换算。**

```
{"serial": 3000, "amount": "100000000"}   →  100,000,000 个 hacds
```

上一版是 decimal 8，要除以 1e8。**这一版不要除。**

4.1 资产身份（用于上币登记）

字段	值
serial	3000
ticket	HACDS
name	hacds
decimal	0
supply	5963298700000000
issuer	T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH

⚠ 链上还有一个 serial 1028 的资产也叫 HACDS（name 是 "HACD Fragments"，别人的）。

按 serial 认，不要按 ticket 认。

4.2 充值识别

扫块，找 TransferAssetFromTo 动作，serial == 3000 且 to == 你的充值地址。

```
GET /query/block/intro?height={H}&tx_hash_list=true
  → 遍历 tx_hash_list，逐个 GET /query/transaction?hash={h}
  → 看 actions 里的 asset 转账
```

建议确认数：3 个块（约 15 分钟）。Hacash 出块目标 300 秒，但会有波动。

4.3 提现构造

提现是标准资产转账：

```
hacds_txbuild transfer {main} {ts} asset-from-to {from} {to} 3000 {amount}
```

> ⚠ **serial 必须写数字 `3000`，不要写 `hacds` 别名。**
> `hacds_txbuild` 里的 `hacds` 别名指向旧版 serial `2000`，与 #3000 版合约的 `HACDS_SERIAL = 3000` 不一致，链上会被 `PermitAsset` 以 `return 1` 拒绝。
> 走 `/api/build` 时服务端会把 `hacds` 自动翻成 `3000`，但**命令行直接调用**不受这层保护。

或者走构建服务（`mode=transfer`，`sub=asset-from-to`）。

4.4 广播与确认

```
POST {node}/submit/transaction
{"tx": "...hex..."}
```

`ret:0` 只代表进内存池。按块高回查 `/query/transaction?hash=`，

`block_height` 出现才算确认。

4.5 手续费与 gas

项	说明
fee 格式	`值:尾数`，如 `0:248`（极小）、`2:247`（0.02 HAC）
手续费公式	`fee = purity × billing_size`
`gasMax`	`1–99`，默认 99。**合约调用不要低于 99**

4.6 三个必知的硬限制

① **单笔交易 16,384 字节。** 这是编译期常量。存取 200 颗钻石的交易接近这个上限。

② **沙箱每 IP 并发 = 2。** 超了直接拒绝，不排队。交易所后端必须串行化。

③ **三个 gas 桶**：compute 18,009 / resource 6,100 / storage 111,911。

超了报 `OutOfGas(31): <桶名> N > M`。**提高 `gasMax` 无效**——那是总额度，

单桶上限是硬约束。

> #3000 之所以能把「单笔 1 颗」提到「单笔 200 颗」，就是因为把算分挪进了 RV64IM 内核：200 颗只花 3,587 compute，一个桶装得下。旧版 FitSH 算 1 颗就要 3,270 resource，2 颗就超 6,100。

4.7 做市 / 报价需要的数据

```
GET {node}/query/contract/sandboxcall?contract={合约}&function=amm_reserves&caller={合约}
GET {node}/query/contract/sandboxcall?contract={合约}&function=amm_reserve_ds&caller={合约}
```

现货价（zhu per hacds）= `amm_reserves × 1e8 / amm_reserve_ds`。

批量拿更省事：

函数	一次拿到
`ref_price_zhu_per_ds`	平滑参考价
`pool_price`	池价（×1e8）
`total_debt` / `utilization`	借贷利用率
`hacd_count` / `hacd_avg_score`	池子规模

4.8 对账

链上恒等式：

```
用户 due 总和 + AMM 池 + 金库累积 + LP 奖励池  ==  链上已发行的 hacds 总量
```

再加一条：

```
有效 HAC 余额总和 × hcut == 名义份额总和
```

索引器的 `GET /backing` 每次同步都会核对这个。对不上就是有 bug。

4.9 上币对接清单

- [ ] 用 serial `3000` 登记，decimal 填 `0`（**不是 8**）
- [ ] 充值只认 serial 3000，忽略同名 ticket 的 #1028
- [ ] 提现用 `asset-from-to`，`serial` 传 `hacds` 或 `3000`
- [ ] 确认数 ≥ 3
- [ ] 后端对沙箱调用串行化（并发上限 2）
- [ ] 大额提现检查交易大小是否撞 16,384 字节上限
- [ ] 对账用 `/backing` 和链上恒等式

---

5. 错误处理

5.1 合约返回码

沙箱调用返回的 `ret_val` 非 0 时，是合约函数内部的 `return N`。

通用

码	含义
`2`	参数格式错（如 HACD 名列表 wire 格式不合法）
`48`	数量越界（存 0 颗 / 超 200 颗 / 赎回不是 1 颗）

码	含义
`5`	评分为 0
`53`	超出总供应上限
`54`	覆盖率不足（100% 发行下恒不触发）
`59`	已过期（`deadline`）
`61`	HACD 池无记录 / 分数不足（赎回）

AMM

码	含义
`2`	某一侧为 0
`23`	池非空但份额归零（防尺度重置）
`24`	份额算出来为 0（比例太小）
`25`	LP 份额不足
`26`	首注单侧超过上限 `1e18`
`56`	滑点保护触发（`min_out` / `min_share` 未达）

借贷

码	含义
`40`	超出可借额度（`(debt+hac) × 200% > coll_hac`）
`41`	债务为 0（还款）
`50`	全局利用率超 50%
`51`	借款已暂停（利用率满 或 价格熔断）
`52`	超过单借款人上限（20%）

清算

码	含义
`42`	该地址无债务
`43`	未达清算线（不可清算）
`44`	清算金额低于最小额（1 HAC）
`45`	抵押产出低于 `min_coll_out`
`46`	深水清算要求全额（`repay < deep_pay`）
`47`	参考价未初始化，拒绝清算

管理员相关的一律不存在

`set_fee` / `set_loan_rate` / `set_liquidate_bonus` / `set_paused` / `set_oracle`

在 #3000 里**没有实现**。调用它们会返回：

```
CallNotExist(51)
```

这不是权限不足，是**函数不存在**。别再写这些调用了。

5.2 节点 / 沙箱错误

按字符串匹配，常见几类：

原始信息	含义	处理
`NotFindContract`	该地址没有合约代码	部署前正常；部署后检查合约地址
`invalid address argument` / `InvalidChecksum`	地址格式或校验位错	重新取完整地址
`balance ... is insufficient`	调用者余额不足付预扣 gas	沙箱调用传 `caller={合约地址}`
`sandbox_per_ip_concurrency_exceeded`	并发超限	串行化 + 退避重试
`resource gas limit exceeded`	resource 桶超限	拆笔；**提高 gas_max 无效**
`compute gas limit exceeded`	compute 桶超限	拆笔
`storage gas limit exceeded`	storage 桶超限	减少新键或降租期
`OutOfGas(31)`	某个桶不够	看括号里的桶名，拆笔
`Arithmetic(90)`	算术下溢/上溢	检查参数；这是合约 bug，请报告
`CallNotExist`	合约没有这个函数	检查函数名（管理员函数已删除）
`state changed`	期间链上状态变了	重试
`transaction parse failed`	交易体解析失败	hex / 字节不完整
`StorageNotActive`	存储键已过期	调 `rent_keys` 续租
`ECONNREFUSED` / `timeout`	服务不可达	检查服务是否在跑

判别要点：这些错误大多是 **HTTP 200 +** `{"ret":1,"err":"..."}`，

只看 HTTP 状态码会漏掉。

---

6. 快速参考

6.1 端点汇总

```
# 全节点
GET  /query/block/height/latest
GET  /query/fee/average
GET  /query/balance?address={A}&assets=true&diamonds=true
GET  /query/diamond?name={N}
GET  /query/contract/sandboxcall?contract={C}&function={F}&params={P}&caller={C}
GET  /query/block/intro?height={H}&tx_hash_list=true
GET  /query/transaction?hash={H}
POST /util/transaction/check        {"tx": "hex"}
POST /submit/transaction            {"tx": "hex"}

# 索引器
GET  /health
GET  /snapshot
POST /sync
GET  /backing
GET  /rent_batches

# 构建服务
GET  /health
POST /api/build
```

6.2 常量

常量	值
合约地址	`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`
合约 nonce	`2`
金库	`1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`
hacds serial	`3000`
hacds decimal	`0`
hacds 总供给	`5963298700000000`
`SCORE_SCALE`	`100000000`（1e8）
`MAX_DEPOSIT_COUNT`	`200`
`FEE_BPS`	`50`（0.5%）
`BORROW_RATIO_BPS`	`20000`（200%）
`LIQ_COLL_RATIO_BPS`	`16000`（160%）
`CLOSE_FACTOR_BPS`	`5000`（50%）
`DEEP_TRIGGER_BPS`	`10500`（105%）
`LIQ_BONUS_BPS`	`500`（5%）

常量	值
`U_MAX_BPS`	`5000`
`LENDABLE_AMM_BPS`	`3000`
VM 三桶	compute 18,009 / resource 6,100 / storage 111,911
交易上限	16,384 字节
gasMax 范围	1–99

6.3 单位换算

HAC（链上 zhu）：

```
HAC = zhu / 1e8
"5:249" = 5 × 10^(249-248) = 50 HAC
"1:248" = 1 HAC
```

hacds（decimal 0）：

```
hacds 个数 == 链上整数，不做换算
```

⚠ **不要对 hacds 除以 1e8。** 那是上一版（decimal 8）的做法。

6.4 一段最小可用的读状态代码

```
const NODE = 'https://api.hacds.com';
const CONTRACT = 'T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH';

async function call(fn, params = '') {
  const q = new URLSearchParams({
    contract: CONTRACT,
    function: fn,
    params,
    caller: CONTRACT,       // 不传会因余额不足失败
  });
  const r = await fetch(`${NODE}/query/contract/sandboxcall?${q}`);
  const j = await r.json();
  if (j.ret !== 0) throw new Error(j.err || 'call failed');
  return j.ret_val;
}

// 池子两侧储备
const hacZhu = await call('amm_reserves');
const ds     = await call('amm_reserve_ds');

// 现货价（zhu per hacds）
const spot = Number(hacZhu) * 1e8 / Number(ds);
```

并发注意：每个 IP 只有 2 个并发名额。批量读要排队，不要 `Promise.all`。

本文档对应合约 `HacdsDeFi3000.fitsh` 定稿版本，日期 2026-11-11。